

Mostenire si abstractizare

Mostenirea creste flexibilitatea programului prin crearea de clase generale din care alte clase isi pot deriva comportamentul si attributele

Clasele din varful ierarhiei se numesc clase de baza sau **superclase**, iar clasele derivate din acestea se numesc **subclase**

In Java orice clasa este derivata din clasa **Object**

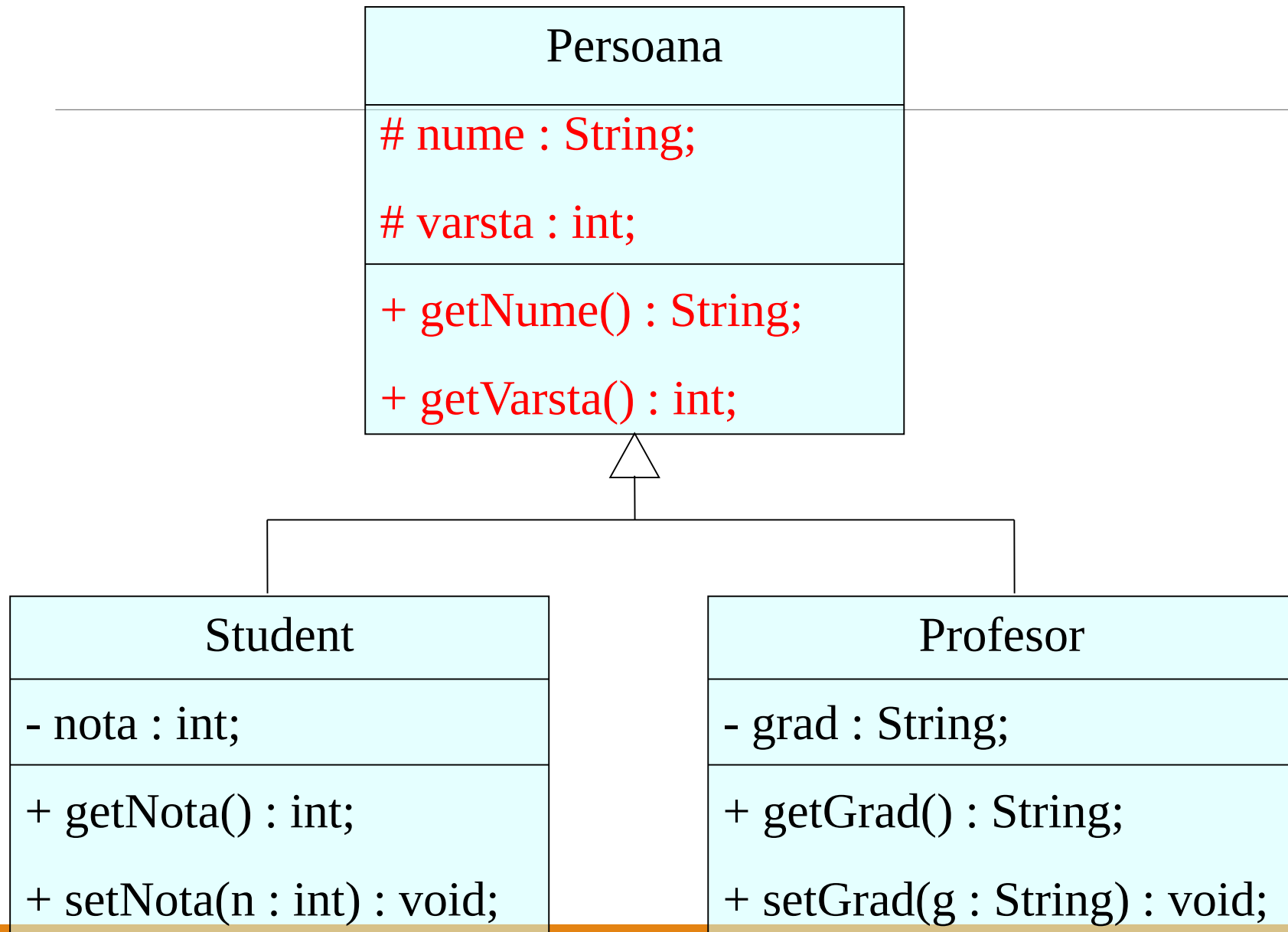
Mostenire si abstractizare

Ex. O clasa Student si o clasa Profesor

Student
- nume : String; - varsta : int; - nota : int;
+ getNume() : String; + getVarsta() : int; + getNota() : int; + setNota(n : int) : void;

Profesor
- nume : String; - varsta : int; - grad : String;
+ getNume() : String; + getVarsta() : int; + getGrad() : String; + setGrad(g : String) : void;

Mostenire si abstractizare



Mostenire si abstractizare

```
public class Persoana {  
    protected String name;  
    protected int varsta;  
    public Persoana(String n, int v) {  
        name = n; varsta = v;  
    }  
    public String getName() {  
        return name;  
    }  
    public int getVarsta() {  
        return varsta;  
    }  
}
```

Mostenire si abstractizare

```
public class Student extends Persoana{
    private int nota;
    public Student(String n, int v) {
        super(n, v);
        nota = 8;
    }
    public int getNota() {
        return nota;
    }
    public void setNota(int n) {
        nota = n;
    }
}
```

Mostenire si abstractizare

Lucrurile comune constituie baza pentru mostenire, ele fiind grupate in superclase

Doua tehnici sunt folosite in realizarea ierarhiilor de clase:

- **Generalizare** – descoperirea comportamentului comun al claselor
- **Specializare** – identificare particularitatilor subclaselor

Mostenire si abstractizare

Mostenire inseamna a deriva noi definitii de clase din clase existente

Motive pentru a folosi mostenirea:

- **Refolosirea** claselor (clasele folosite mai mult sunt si testate mai mult, deci mai fiabile)
- **Standardizarea** comportamentului unui grup de clase
- **Polimorfismul** (folosirea aceluiasi cod pentru clase diferite)

Mostenire si abstractizare

Subclasele mostenesc toate attributele si metodele **neprivate** ale clasei de baza

Subclasele pot adauga attribute si metode

Subclasele pot redefini metode

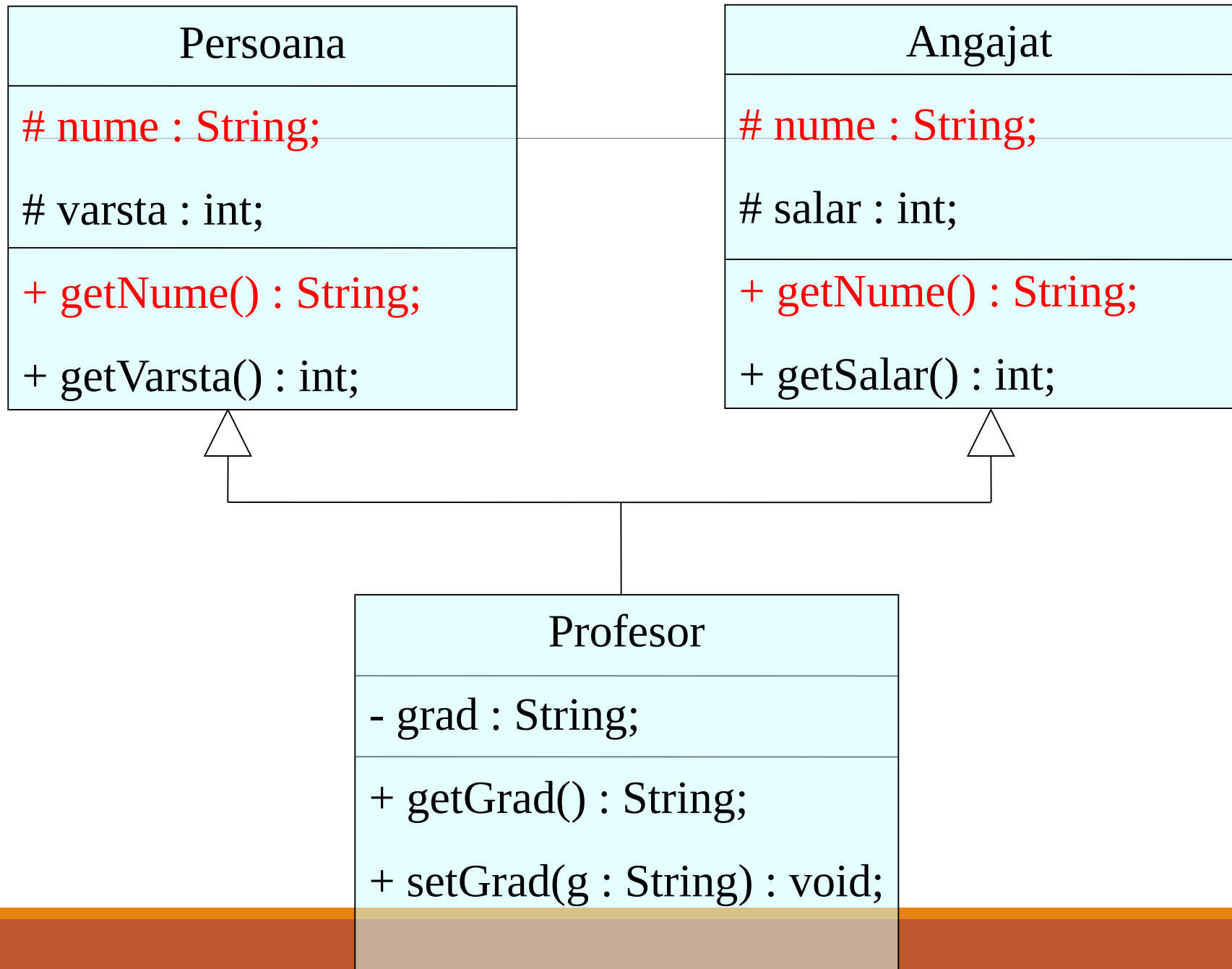
Constructorii nu sunt mosteniti niciodata

Un obiect dintr-o subclasa poate inlocui orice obiect dintr-o clasa de baza pe care o mosteneste

Mostenire : Cuprins

1. Mostenire si abstractizare
2. **Problema mostenirii multiple**
3. Modificatorii de acces si mostenirea
4. Constructorii si mostenirea

Mostenirea multipla



Mostenirea multipla

In Java o clasa nu poate extinde mai multe clase

Daca o subclasa ar extinde doua clase de baza, si in cele doua clase de baza ar exista metode sau attribute identice, subclasa nu ar sti pe care dintre ele sa le mosteneasca

Mostenire : Cuprins

1. Mostenire si abstractizare
2. Problema mostenirii multiple
3. **Modificatorii de acces si mostenirea**
4. Instanțe predefinite
5. Constructorii si mostenirea

Modificatori de acces

Orice clasa are acces la attributele / metodele proprii

Daca un atribut / metoda este declarat **public**, atunci orice alta clasa **are** acces la el

Daca un atribut / metoda este declarat **private**, atunci orice alta clasa **nu are** acces la el

Modificatori de acces

Daca un atribut / metoda nu are modificador de acces (default, package), atunci orice alta clasa din acelasi pachet **are** acces la el, si orice clasa din alt pachet **nu are** acces la el

Daca un atribut / metoda este declarat **protected**, atunci orice alta clasa derivata sau din acelasi pachet **are** acces la el, si orice clasa nederivata din alt pachet **nu are** acces la el

Modificatori de acces

Tabela completa a drepturilor de acces in Java

Modificator	Clasa din acelasi pachet	Clasa din alt pachet	Subclasa din acelasi pachet	Subclasa din alt pachet
public	DA	DA	DA	DA
private	NU	NU	NU	NU
protected	DA	NU	DA	DA
(default, package)	DA	NU	DA	NU

Modificatori ai claselor

Sunt folositi pentru a schimba unele caracteristici ale claselor. Se aplică înainte de numele clasei în momentul declarării acesteia. Există două categorii de modificatori: de **acces** și de **tip**.

Modificatorii de acces sunt folosiți pentru stabilirea modului intern și extern de utilizare a clasei. De fapt se stabilește gradul de vizibilitate al unor porțiuni sau al întregului obiect precum și ale metodelor acestuia

Modificatorii de tip sunt folosiți pentru specificarea categoriei din care face parte adică este o clasă propriu-zisă sau este un model pentru derivarea altor clase

Modificator de tip - public

Există cazuri în care este necesar ca să avem un grad mai mare de acces la caracteristicile unei descrieri. În acest caz se folosește modificatorul public

Prin declararea unei clase ca fiind de acest tip se permite celorlalte clase sau pachete de clase să îi acceseze atât variabilele și metodele cât și subclasele. Iată un exemplu de clasă publică:

```
public class Câine
{
    public static void main(String arg[])
    {
        System.out.println("Un câine oarecare")
    }
}
```

Modificator de tip - Abstract

Clase abstracte

Reprezintă cel mai înalt nivel de abstractizare. Din aceste clase se vor deriva celelalte care fac parte din aceeași categorie.

Cu cât sunt mai sus într-o ierarhie de clase, **superclasele** devin tot mai generale și abstracte

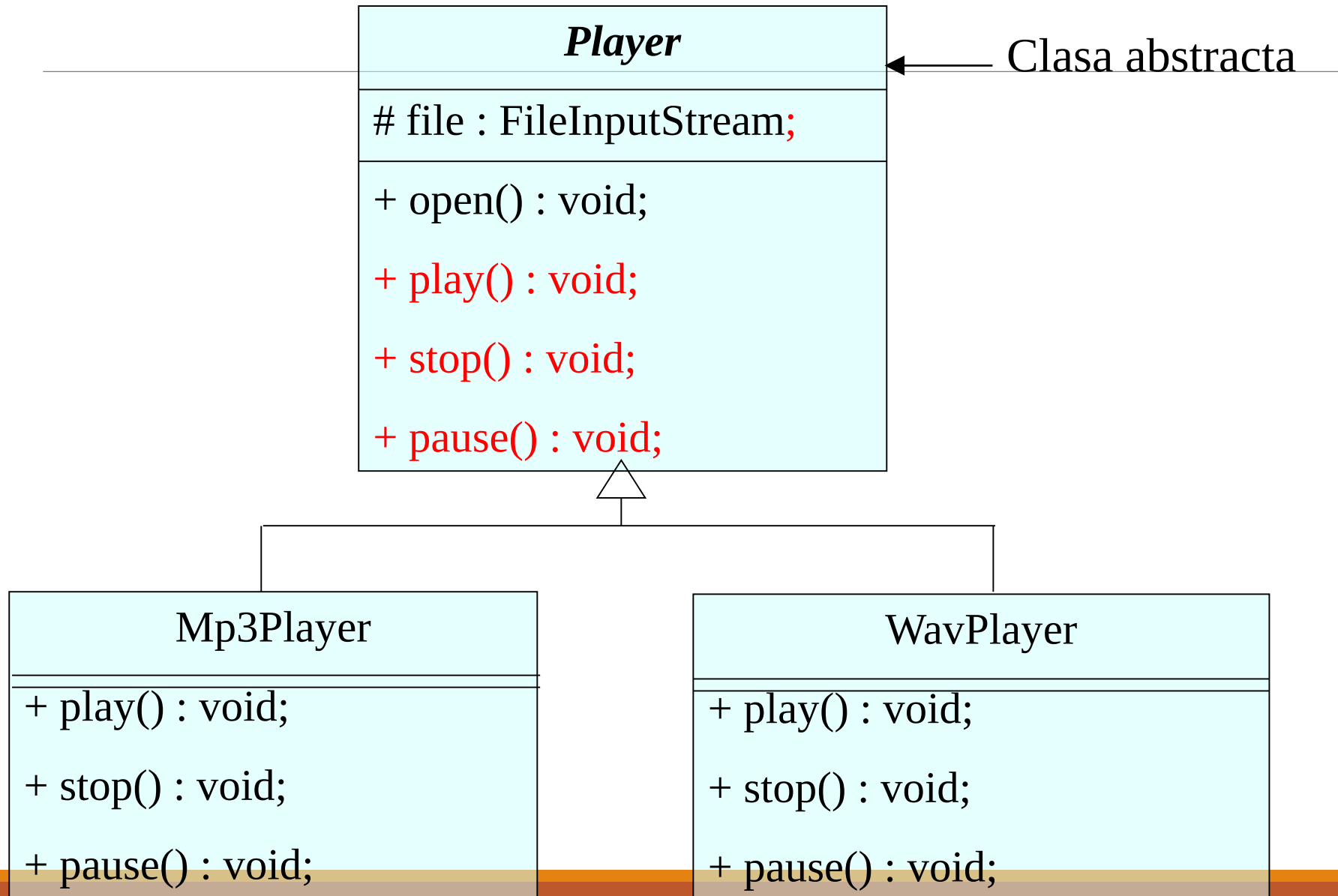
La un moment dat unele superclase sunt mai degrabă un **model** pentru alte clase decât o definiție a unor obiecte

În acest caz superclasele sunt declarate **abstracte** și nu se pot crea obiecte din ele

Există situații în care se dorește declararea unei clase dar nu e clar cum se vor defini toate metodele acelei clase.

De exemplu dacă se dorește realizarea clasei mamifer care să aibă obligatoriu o metodă de marcarea teritoriului nu se definește la acest nivel respectivă metodă ci doar la nivel de clasă derivată

Clase abstracte



Clase abstracte

```
public abstract class Player {  
    protected FileInputStream file;  
  
    public void open() {  
        file = new FileInputStream ...  
        ...  
    }  
  
    public abstract void play();  
  
    public abstract void stop();  
  
    public abstract void pause();  
}
```

Clase abstracte

O **metoda abstracta** este o metoda fara corp (continut)

O clasa care contine o metoda abstracta, trebuie declarata **clasa abstracta**

Nu se pot crea obiecte dintr-o clasa abstracta

O clasa poate fi declarata abstracta si daca nu contine nici o metoda abstracta

Modificator de tip - Abstract

Iată un exemplu de particularizări posibile pornind de la un model general:

```
public abstract class Mamifer
{
    abstract void MarcheazăTeritoriul()
}

public class Om extends Mamifer
{
    public void MarcheazăTeritoriul()
    {
        // construiește un gard
    }
}
```

```
public class GolanDeCartier extends
Mamifer
{
    public void MarcheazăTeritoriul()
        { // desenează pe pereți
        }
}

public class Câine extends Mamifer
{
    public void MarcheazăTeritoriul()
        { // știe toată lumea cum...
        }
}
```

Modificator de tip - Abstract

Acest tip de clasă respectă o serie de reguli:

- din ea nu se poate instantia nici un obiect
- trebuie să conțină cel puțin o declarație variabilă sau de metodă abstractă
- Toate metodele abstracte trebuie să conțină implementări efective în clasele derivate
- Nu pot fi declarate ca de tip final sau privat

Modificator de tip - final

Au apărut din necesitatea de a menține sub control dezvoltarea modelelor peste o anumită dimensiune.

O clasă de acest tip se consideră a fi ultima derivată și nu mai poate fi extinsă.

final aplicat la o variabila sau atribut îl face constant; după ce a primit o valoare nu mai poate fi modificat

final aplicat la o metodă face ca metoda să nu mai poate fi redefinită

final aplicat la o **clasa** face ca să nu se mai poate deriva alte clase din ea

Constuctorii si mostenirea

Constructorii nu sunt mosteniti ca celelalte functii

Atunci cand nu se declara nici un constructor, compilatorul insereaza un constructor default (null constructor)

Daca se declara cel putin un constructor, nu se mai genereaza automat null constructor

Constructorii sunt supraincarcati (overloaded – mai multi constuctori cu acelasi nume)

Constuctorii si mostenirea

Prima data se apeleaza constructorul superclasei si apoi al subclasei

In constructorul unei subclase prima linie de cod poate fi un apel la un constructor din clasa de baza, de forma

super(<argumente>);

Daca nu exista aceasta linie se apeleaza automat constructorul default al clasei de baza daca el exista

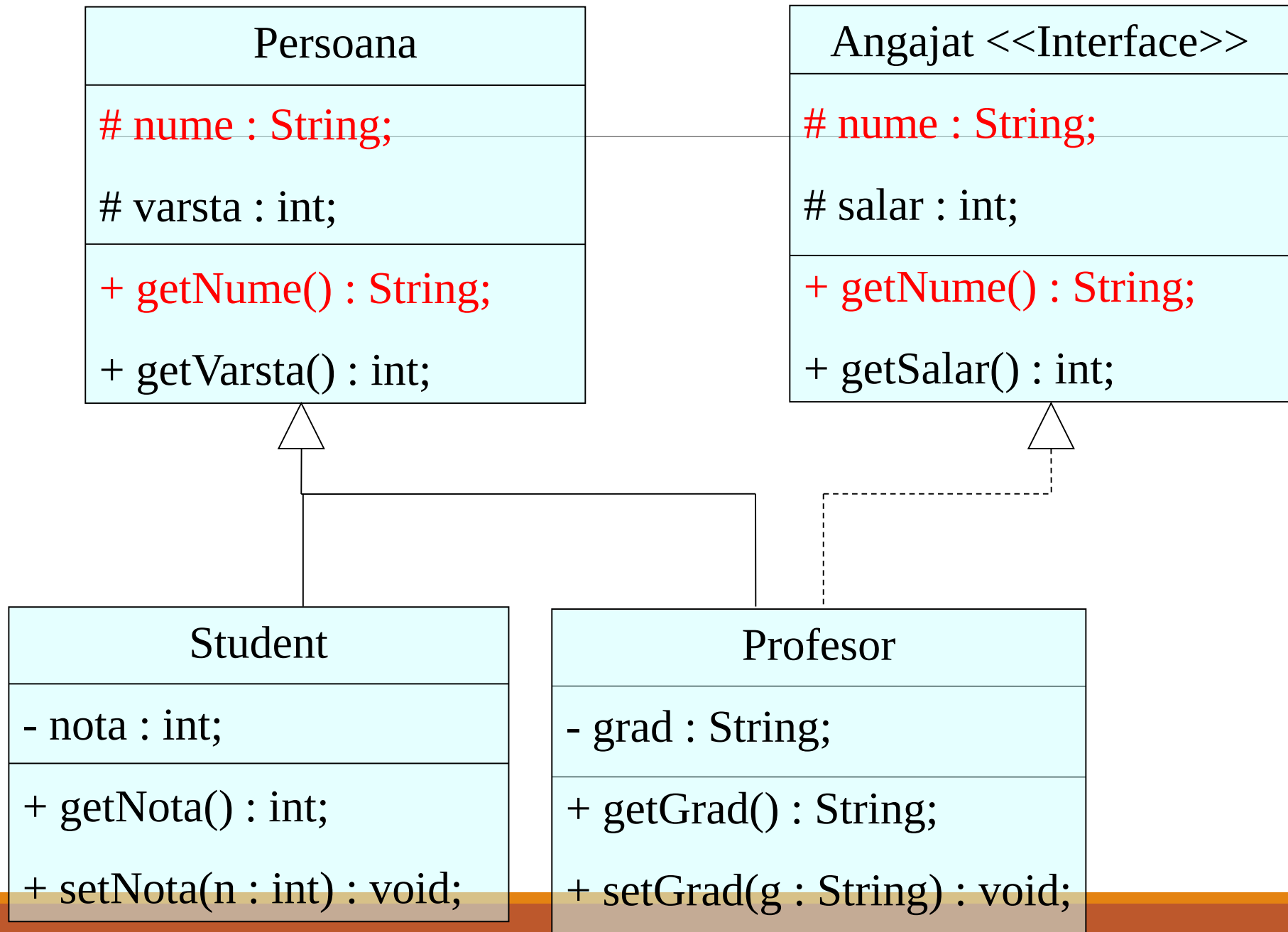
Interfete

O **interfata** este o pseudo-clasa complet abstractă si poate contine doar metode abstracte si constate

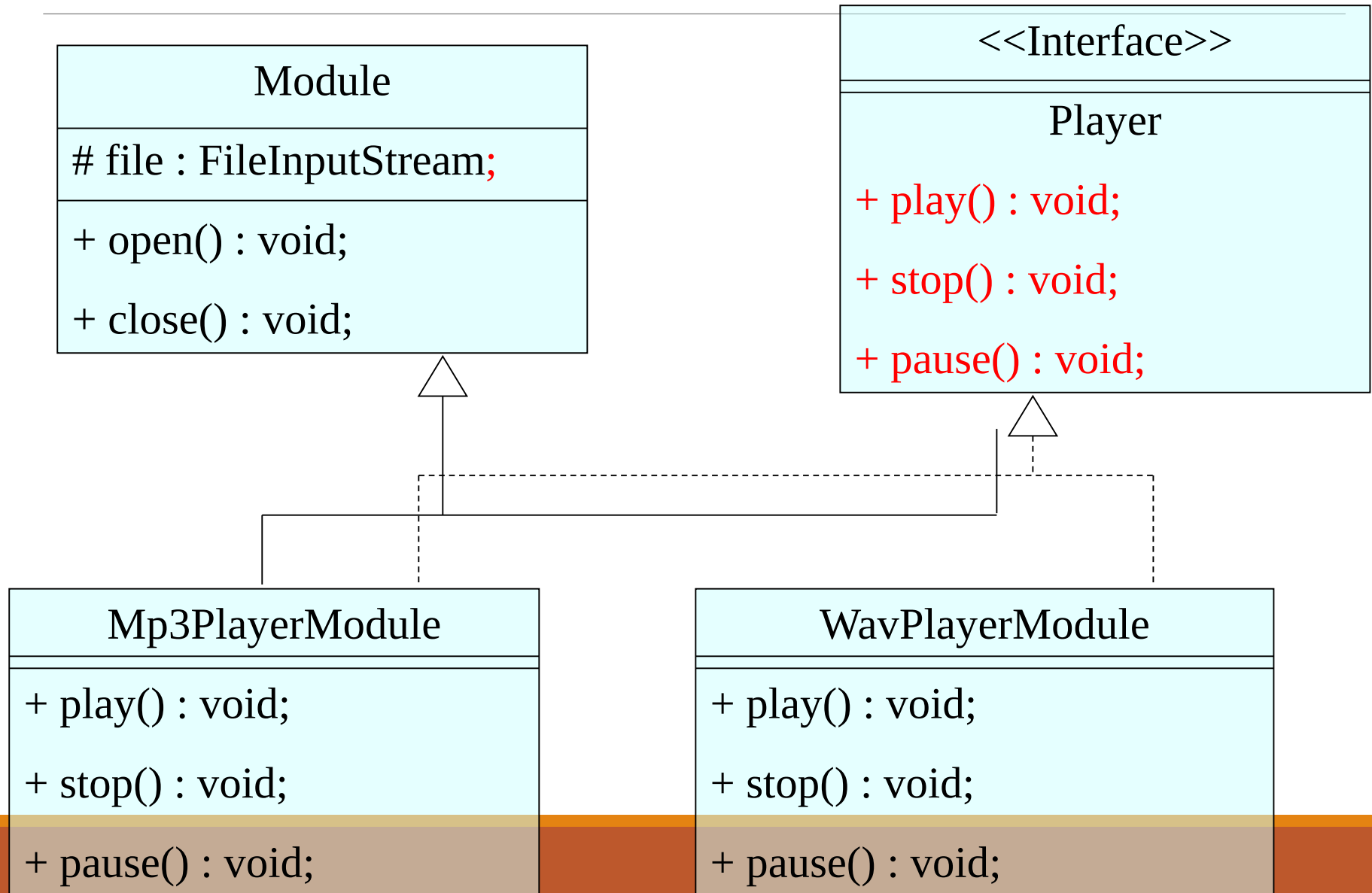
O interfata este un **contract** pe care alte clase se angajeaza sa il implementeze

O clasa poate extinde o singura clasa dar in acelasi timp poate **implementa** oricate interfete

Interfețe



Interfete



Interfete

```
public interface Player {  
  
    public void play();  
  
    public void stop();  
  
    public void pause();  
}
```

Interfete

```
public class Mp3PlayerModule extends Module implements Player {  
  
    public void play() {  
        ...  
    }  
  
    public void stop() {  
        ...  
    }  
  
    public void pause() {  
        ...  
    }  
}
```

Interfete

Toate metodele unei interfete sunt automat abstracte si trebuie declarate publice

O interfata poate declara constante, care sunt automat publice, statice si finale

Interfetele pot extinde alte interfete

Mai multe clase neinrudite pot implementa aceeasi interfata

O interfață descrie un model comportamental generic care poate fi implementat de orice subclasă din ierarhia asociată.

O interfață definește o serie de metode dar nu și plementarea acestora

O clasă derivată dintr-o interfață va implementa și metodele definite în interfață pe lângă cele proprii

Într-o interfață se pot defini și variabile. Acest lucru este necesar deoarece un model poate presupune și o stare inițială sau de echilibru

Într-o interfață nu se pot folosi următorii modificatori: transient, volatile sau synchronized și de asemenea nici private sau protected

Interfețe

Diferențe între clase și interfețe

- Interfețele nu implementează metodele
- o clasă poate aparține (implementa) mai multor interfețe (o formă de mostenirea multiplă) dar poate avea o singură superclasă
- O interfață este o descriere formală de sine stătătoare pe când o clasă trebuie să aparțină unei ierarhii unice de clase. E permis însă ca, clase din ierarhii diferite să aparțină (implementeze) aceeași interfață

Observație: termenul de **implementare** este abordarea din punct de vedere practic în timp ce termenul de aparține este din punct de vedere formal folosit